MODULUS

Product Requirements Document

For Dev Team & LLM Context

Next.js · Supabase · Cloudflare R2 · Socket.io · WebRTC · Excalidraw · Recharts

All Features — Single Release

1. Product Overview

1.1 Problem Statement

College students lack a single cohesive platform for academic collaboration. Resources scatter across WhatsApp, Google Drive, and Notion. There is no shared accountability layer, no structured community organisation, and real-time study collaboration requires juggling Discord, Zoom, and separate whiteboard tools simultaneously.

1.2 What MODULUS Is

MODULUS unifies four core workflows into one product:

- Personal Vault — private storage for all notes, files, and links. The single source of truth for every file on the platform.
- Community Collaboration — shared resource libraries (referencing Vault files), task accountability, and discussion threads per course or group.
- Real-Time Study Circles — ephemeral rooms with voice, video, screen share, and a shared Excalidraw whiteboard.
- Focus Dashboard — Pomodoro timer, session logging, community leaderboard, and personal analytics charts.

1.3 Target Users

Persona	Description	Primary Pain
The Note Sharer	Organised student who creates good notes and wants community visibility	No easy way to share without losing personal copy or organisation
The Collaborative Learner	Works best in groups; needs real-time tools built for study, not meetings	Juggling 3-4 apps simultaneously kills focus
The Accountability Seeker	Needs group progress visibility to stay disciplined	No shared task or focus layer in existing student tools
The Community Builder	Course rep or study group leader managing a cohort	No role-based moderation in student platforms

1.4 Success Metrics

Metric	Definition	Indicative Target
Community Creation Rate	% of users who create at least one Community	≥ 30%

Metric	Definition	Indicative Target
Vault Engagement	% of users who upload at least one file	≥ 60%
Study Circle Adoption	% of communities hosting at least one Circle per week	≥ 40%
D7 Retention	Users returning within 7 days of signup	≥ 50%
Task Completion Rate	% of Common Tasks completed by ≥ 50% of members	≥ 55%

2. Scope

Approach	All features ship in a single release. No MVP/v2 split. Build everything listed below.

Module	Included
Authentication	Email/password + Google OAuth (Supabase Auth), email verification, password reset, JWT sessions, skippable profile wizard
Personal Vault	Upload files/notes/links, folders + sub-folders, #tags, search, quota enforcement (500 MB / 20 MB per file), multi-community sharing, versioning (latest state), save-to-vault copies (independent R2 copy)
Community Core	Public/Private, roles (Owner/Curator/Peer), join requests, member cap (250), search/discovery
Community Vault	Share from Vault, direct upload (auto-saves to Vault), community search, view/download, save to own Vault (full copy), community folders (independent of personal structure)
Collaborative Tasks	Common Tasks, peer suggestions, completion tracking, progress bar, drill-down view with opt-out
Discussion Threads	Post types, threaded replies, upvote/downvote, lock/delete/mark solution, moderation by Owner/Curator
Focus Dashboard	Pomodoro timer (25/5), stopwatch, session logging with labels, community leaderboard, personal analytics (Recharts bar/line charts)
Study Circles	Full-mesh WebRTC max 5 (voice/video/screen share), ephemeral text chat, Excalidraw whiteboard with snapshot-to-Vault, Open Relay TURN
System Admin	Global dashboard, god-mode access, force-delete any entity

2.1 Explicitly Out of Scope

- Notifications of any kind — no in-app, no email, no push.
- Flashcard decks.
- Mobile native apps (responsive web only).
- Direct messaging between users.
- AI features (summarisation, recommendations, etc.).
- Payments or premium tiers.

3. Full Technology Stack

Layer	Technology	Purpose
Framework	Next.js 14 (App Router)	Frontend + API Route Handlers. Replaces separate Express server.
Database	Supabase (PostgreSQL)	Managed Postgres. Row Level Security enforced at DB level.
Auth	Supabase Auth + @supabase/ssr	Email/password + Google OAuth. JWT sessions compatible with Next.js App Router.
File Storage	Cloudflare R2 + @aws-sdk/client-s3	Object storage. Zero egress fees. Short-TTL signed URLs server-side.
WebSocket	Socket.io (separate Node.js server)	Study Circles only: WebRTC signalling, Excalidraw sync, ephemeral text chat.
WebRTC	Browser-native RTCPeerConnection	Peer-to-peer voice, video, screen share. Full-mesh, max 5 peers.
Whiteboard	Excalidraw (@excalidraw/excalidraw)	Embedded React whiteboard. onChange deltas via Socket.io. exportToBlob() for snapshots.
TURN	Open Relay by Metered	Free TURN server. NAT traversal fallback for WebRTC.
UI Components	shadcn/ui (Radix UI)	Accessible, unstyled component primitives styled with Tailwind.
Styling	Tailwind CSS	Utility-first CSS. Co-located with JSX.
Icons	Lucide React	Clean icon set. Used natively by shadcn.
Animation	Framer Motion	Page transitions, modal animations, UI polish.
Forms	React Hook Form + Zod	Form state + validation. Zod schema shared client and server.
Server State	TanStack Query	Data fetching, caching, background refetch, optimistic updates.

Layer	Technology	Purpose
Client State	Zustand	Circle room state, UI panels, active timer state.
Charts	Recharts	Bar and line charts for Focus Dashboard personal analytics.
TypeScript	TypeScript (full)	Type safety. Supabase types auto-generated from schema.

Socket.io Deployment	Next.js (Vercel or similar) and Socket.io server (Railway or Render free tier) are deployed separately. Frontend connects via NEXT_PUBLIC_SOCKET_URL env variable. Socket.io server is ~50 lines of Node.js — just room management and event relay.

4. Key User Flows

4.1 Onboarding

#	Action	System
1	Register or Sign in with Google	Email: hash password, send verification email. Google: OAuth flow via Supabase, skip verification.
2	Email verification (email users only)	Account inactive until link clicked. Resend option. Google users skip.
3	Profile Setup Wizard	Profile pic, college, stream, course/year, interest tags. Skip button on every step.
4	Land on Dashboard	Empty state with CTAs: Create a Community, Explore Communities.

4.2 Vault Upload & Share

#	Action	System
1	Upload file or create note	Client: check size ≤ 20 MB and quota < 500 MB. Server: hard reject if either fails. File stored once in R2.
2	Organise	Add to folder, assign #tags.
3	Share to Community	Select Community (and optional Community folder). File becomes visible to all members. One R2 object, new DB reference.
4	Edit note	Changes saved to single DB record. Auto-updates in all Communities it is shared to.

#	Action	System
5	Unshare or Delete	Unshare: removes from Community, Vault copy untouched. Delete: removes from all Communities. R2 object deleted only if no other user saved a copy.

4.3 Community Member Saves a File

#	Action	System
1	Member views a shared file in Community Vault	File displayed with View, Download, and Save to My Vault options.
2	Member clicks Save to My Vault	Full R2 copy created under member's ownership. New VaultItem record created. Counts against member's 500 MB quota.
3	Original owner later deletes their file	Owner's VaultItem and Community reference deleted. R2 object preserved for member's copy. Member's file unaffected.

4.4 Study Circle Session

#	Action	System
1	Member starts Circle	Socket.io room created (keyed by Circle ID). Circle record saved to Supabase. Appears as 'Live' in Community.
2	Others join (up to 5)	Full-mesh WebRTC handshake via Socket.io signalling. 6th joiner gets 'Room Full' error.
3	Collaborate	Voice/video/screen share via WebRTC. Text chat via Socket.io. Excalidraw onChange deltas synced via Socket.io to all participants.
4	Save Snapshot	exportToBlob() → PNG uploaded to R2 → VaultItem created in user's Vault.
5	Last user leaves	Socket.io room destroyed. Text chat wiped from memory. Circle record deleted from Supabase.

4.5 Focus Session

#	Action	System
1	User opens Focus Dashboard in a Community	Shows Pomodoro timer, stopwatch, leaderboard, and personal analytics chart.
2	Select label and start timer	Label (e.g. 'DSA') required before start. Countdown begins.
3	Session ends or user stops manually	Duration logged to FocusSession table with user_id, community_id, label, duration_seconds, started_at.

#	Action	System
4	Leaderboard updates	Community leaderboard re-ranks based on cumulative session hours.
5	Personal analytics	Recharts chart renders daily/weekly breakdown by label from FocusSession records.

5. User Stories & Acceptance Criteria

5.1 Authentication

ID	User Story	Acceptance Criteria
US-01	As a new user, I want to register with email and password	• Name, Email, Password (≥8 chars) required. • Duplicate email shows 'Account already exists'. • Verification email sent on submit. Account inactive until verified. • Inline validation errors shown per field.
US-02	As a new user, I want to sign up or log in with Google	• 'Sign in with Google' on both login and signup screens. • OAuth completes, account created or logged in. • Google users skip email verification. • If Google email already registered via password, accounts linked.
US-03	As a user, I want to reset my forgotten password	• Reset link sent to registered email. Expires 1 hour. • New password cannot match current password. • Confirmation shown after successful reset.

5.2 Personal Vault

ID	User Story	Acceptance Criteria
US-04	As a student, I want to upload a file to my Vault	• Accepted types: PDF, PNG, JPG, JPEG, DOC, DOCX. • File > 20 MB: error shown before upload starts. • Upload blocked if user quota ≥ 500 MB. • Upload progress indicator shown. File appears in Vault on completion.
US-05	As a student, I want to organise files with folders and tags	• Create, rename, delete folders and sub-folders. • Assign one or more #tags to any file or note.

ID	User Story	Acceptance Criteria
		• Vault search returns results matching filename, heading, or tag.
US-06	As a student, I want to share a Vault file to a Community	• Share dialog: choose target Community, optional Community folder. • File visible to all Community members immediately. • Editing the note later auto-updates the Community-visible version. • File can be shared to additional Communities without re-uploading.
US-07	As a student, I want to unshare or delete a Vault file	• Unshare: removes from that Community. File remains in Vault. • Delete: removes from all Communities and Vault. R2 object deleted only if no other user saved a copy.

5.3 Community

ID	User Story	Acceptance Criteria
US-08	As a student, I want to create a Community	• Set Name, Description, Banner, and type (Public/Private). • Creator auto-assigned Owner role. • Community appears in search immediately.
US-09	As a Community member, I want to find and join a Community	• Search by name or category tag. • Public: join immediately. Private: send join request, wait for approval. • Join blocked if community at 250 member cap.
US-10	As a Community member, I want to save another member's shared file	• 'Save to My Vault' button on every Community Vault item not owned by viewer. • Creates full R2 copy under viewer's ownership. Counts against viewer's quota. • Item appears in viewer's Vault. Unaffected by original owner's future actions.
US-11	As a Peer, I want to track task completion	• Common Tasks shown with checkbox and community-wide progress bar. • Checking marks task done for current user only. • Drill-down shows names of completers unless member opted out.

5.4 Discussion Threads

ID	User Story	Acceptance Criteria
US-12	As a Peer, I want to post a question in a Community	• Post types: plain text, question/doubt, image attachment. • Threaded replies to any depth. • Upvote/downvote on threads and replies.
US-13	As a thread author, I want to manage my thread	• Lock thread (no new replies, shown as Closed). • Delete thread permanently. • Mark one reply as Solution — visually highlighted.

5.5 Focus Dashboard

ID	User Story	Acceptance Criteria
US-14	As a student, I want to run a Pomodoro or stopwatch session	• Must select a Label/Subject before starting. • Pomodoro: 25-min countdown auto-transitions to 5-min break. • Stopwatch: free-form start/pause/reset. • Session logged on completion or manual stop.
US-15	As a student, I want to see how I compare with my Community on focus hours	• Community Leaderboard shows members ranked by total hours focused in this Community. • Updates after each logged session. • Personal analytics chart (Recharts) shows daily/weekly hours by subject label.

5.6 Study Circles

ID	User Story	Acceptance Criteria
US-16	As a Peer, I want to start a Study Circle	• Circle appears as 'Live' in Community interface immediately. • Creator joins automatically as first participant.
US-17	As a Circle participant, I want to communicate in real time	• Voice via WebRTC. Mute toggle. • Video via WebRTC. Off by default, toggleable. • Screen share via getDisplayMedia. One active at a time. • Text chat via Socket.io. Messages visible to all in room.
US-18	As a Circle participant, I want to collaborate on a shared whiteboard	• Excalidraw canvas visible and editable by all participants simultaneously. • Changes sync in real time via Socket.io. • All Excalidraw tools available: pen, shapes, text, eraser, colour, undo/redo.

ID	User Story	Acceptance Criteria
US-19	As a Circle participant, I want to save the whiteboard before leaving	• 'Save Snapshot' button always visible. • PNG saved to R2 and VaultItem created in user's Vault instantly. • Multiple snapshots allowed per session.
US-20	As a Circle participant, I want to have the room clean up after the session	• Last user leaves: Socket.io room destroyed, all text chat wiped. • Circle record deleted from Supabase. No recovery path.

6. Data Model Summary

Core entities, key fields, and relationships. Intended as implementation context.

Entity	Key Fields	Notes
User	id, name, email, password_hash, google_id, profile_pic, college, stream, course, year, tags[], storage_used_bytes	google_id nullable. storage_used_bytes updated on every upload/delete/save-copy.
Community	id, name, description, banner_url, type, owner_id, member_count	member_count cached, capped at 250.
CommunityMember	community_id, user_id, role (owner/curator/peer), joined_at	Junction table. Role changes update this record.
File	id, owner_id, r2_object_key, filename, mime_type, size_bytes, created_at	One DB record per unique R2 object (binary). r2_object_key is the R2 path.
VaultItem	id, file_id (nullable), owner_id, item_type (file/note/link), note_content, is_private, folder_id, tags[], created_at	file_id null for rich text notes. note_content null for file/link types.
CommunityVaultItem	id, vault_item_id, community_id, community_folder_id, shared_by, shared_at	Reference only. Deleting this does not touch VaultItem or File.
Folder	id, owner_id, name, parent_id, scope (vault/community), community_id	Unified folder table. scope distinguishes personal vs community.

Entity	Key Fields	Notes
Task	id, community_id, title, status (pending/active), created_by, created_at	pending = suggested by Peer. active = approved by Owner/Curator.
TaskCompletion	task_id, user_id, completed_at, show_in_drilldown (bool)	show_in_drilldown = user's opt-in/out setting.
Thread	id, community_id, author_id, content, post_type, is_locked, solution_reply_id, created_at	solution_reply_id FK to Reply.
Reply	id, thread_id, author_id, parent_reply_id, content, created_at	parent_reply_id nullable (top-level replies).
Vote	id, user_id, target_type (thread/reply), target_id, value (+1/-1)	Upsert on change. One vote per user per target.
FocusSession	id, user_id, community_id, label, duration_seconds, timer_type (pomodoro/stopwatch), started_at	Logged on session end. Powers leaderboard and analytics.
StudyCircle	id, community_id, started_by, started_at, participant_count	Deleted when participant_count = 0.
JoinRequest	id, community_id, user_id, status (pending/approved/rejected), reviewed_by, created_at	Deleted after resolution.

7. API Surface

All endpoints are Next.js Route Handlers (REST). Auth via Supabase JWT. Socket.io events handled on the separate Node.js server.

7.1 REST Endpoints

Group	Base Path	Key Endpoints
Auth	/api/auth	POST /register, POST /login, GET /google (OAuth), POST /logout, POST /reset-password
Users	/api/users	GET /me, PATCH /me, GET /me/quota
Vault	/api/vault	POST /items (upload/create), GET /items (list+search), PATCH /items/:id, DELETE /items/:id, POST /folders, GET /folders

Group	Base Path	Key Endpoints
Share	/api/vault/items/:id/share	POST (share to community), DELETE /:communityId (unshare)
Communities	/api/communities	POST / (create), GET / (search), GET /:id, PATCH /:id, DELETE /:id
Members	/api/communities/:id/members	GET /, POST /join-request, PATCH /join-requests/:reqId, DELETE /:userId, PATCH /:userId/role
Community Vault	/api/communities/:id/vault	GET /items (search shared), POST /upload (direct upload), POST /save/:itemId (save to own vault), POST /folders, PATCH /items/:itemId/folder
Tasks	/api/communities/:id/tasks	GET /, POST /, PATCH /:taskId (approve/reject/edit), POST /:taskId/complete, DELETE /:taskId/complete
Threads	/api/communities/:id/threads	GET /, POST /, GET /:threadId, DELETE /:threadId, PATCH /:threadId
Replies	/api/threads/:id/replies	GET /, POST /, DELETE /:replyId
Votes	/api/votes	POST / (upsert on thread or reply)
Focus	/api/communities/:id/focus	POST /sessions (log session), GET /leaderboard, GET /me/analytics
Circles	/api/communities/:id/circles	GET / (active), POST / (start), DELETE /:circleId
Admin	/api/admin	GET /communities, DELETE /entities/:type/:id

7.2 Socket.io Events (Study Circles)

Event	Direction	Purpose
circle:join	Client → Server	Join a Circle Socket.io room
circle:leave	Client → Server	Leave; triggers cleanup if last participant
rtc:offer	Client → Server → Client	WebRTC offer relay between peers
rtc:answer	Client → Server → Client	WebRTC answer relay between peers
rtc:ice	Client → Server → Client	ICE candidate relay between peers
circle:chat	Client → Server → Room	Text chat message broadcast to room
whiteboard:delta	Client → Server → Room	Excalidraw scene delta synced to all participants
circle:participant-update	Server → Room	Participant joined/left, current count update

8. Non-Functional Requirements

Category	Requirement
Performance	API responses (non-file) < 500 ms under normal load. File uploads streamed — must not block other requests.
Security	Passwords hashed via Supabase Auth (bcrypt). JWTs signed by Supabase. R2 files served via short-TTL signed URLs only — bucket never public. Zod validation on all API route inputs.
RLS	Supabase Row Level Security policies enforce data access at DB level. API is a second layer, not the only layer.
Storage	Quota enforced server-side. Client check is UX only and cannot be bypassed. R2 objects deleted when reference count = 0.
Responsiveness	Fully usable on mobile viewports ≥ 375 px. No horizontal scroll.
Reliability	Study Circle cleanup idempotent — server restart deletes orphan Circle records. R2 provides file durability.
WebRTC	TURN server (Open Relay) configured for all RTCPeerConnection instances. Direct connection attempted first; TURN used as automatic fallback.
Privacy	Vault content never exposed to other users unless explicitly shared. Task drill-down respects per-user opt-out setting.

9. Frontend Component Architecture

Component / Route	Responsibility
<App /> / layout.tsx	Root layout. Supabase auth provider. TanStack Query client. Zustand store init.
<AuthLayout /> → /login, /signup, /reset	Unauthenticated pages. Google OAuth button. Email forms with React Hook Form + Zod.
<ProfileWizard /> → /setup	Skippable multi-step onboarding. Step state persisted across navigation.
<AppShell />	Persistent layout: sidebar nav, user avatar. Wraps all authenticated routes.

Component / Route	Responsibility
<VaultPage /> → /vault	Folder tree, file grid/list, upload button, #tag filter, search bar.
<VaultItemModal />	Preview, edit (rich text), share-to-community, unshare, delete.
<ExplorePage /> → /explore	Community search. Filter by category tags.
<CommunityLayout /> → /c/:id	Community shell. Sub-nav: Vault | Tasks | Threads | Focus | Circles.
<CommunityVault />	Shared file grid. Folder nav. Upload / Share button. View, Download, Save-to-Vault per item.
<TasksPanel />	Common Task list. Checkboxes. Progress bars. Drill-down modal.
<ThreadsPanel />	Thread feed. Composer. Thread detail with nested replies and votes.
<FocusDashboard />	Timer selector (Pomodoro/Stopwatch). Label picker. Community leaderboard. Recharts analytics.
<CirclesPanel />	Active Circles list. Start Circle button.
<StudyCircleRoom /> → /c/:id/circle/:circleId	Full-screen room. Video grid. Text chat sidebar. Excalidraw whiteboard. Controls: mute, camera, screen share, snapshot, leave.
<Whiteboard />	Excalidraw component. Emits onChange deltas to Socket.io. Receives and applies remote deltas.
<SettingsPage /> → /settings	Profile edit. Storage quota display. Password change.
<AdminDashboard /> → /admin	Global community list. Force-delete controls.

10. Recommended Build Order

Principle	Each phase is independently testable. Never spend more than one phase on infrastructure nobody can interact with yet.

Phase 1 — Foundation

- Supabase project setup: tables, RLS policies for all entities.
- Next.js app scaffold: routing, Supabase client, auth context (@supabase/ssr), protected routes.
- Supabase Auth: email/password registration + verification + password reset.
- Google OAuth: configure in Supabase dashboard, implement in app.
- Cloudflare R2: bucket setup, upload API route, signed URL generation, delete helper.
- Profile Setup Wizard.

Phase 2 — Personal Vault

- File upload with quota enforcement (client + server).
- Rich text note creation and editing.
- Folders, sub-folders, #tags.
- Vault UI: grid/list view, search, preview modal, delete, unshare controls.

Phase 3 — Communities & Community Vault

- Community creation (Public/Private), search, discovery.
- Role system: Owner, Curator, Peer. Join request flow.
- Community Vault: share from Vault, direct upload (auto-saves to Vault), community search, view/download, save-to-own-vault (full copy).
- Community folders (independent of personal Vault structure).
- Member management: promote, demote, kick, ban.

Phase 4 — Collaboration Features

- Common Tasks: create, suggest, approve, complete, progress bar, drill-down with opt-out.
- Discussion Threads: post types, threaded replies, upvote/downvote, lock, mark solution, moderation.

Phase 5 — Focus Dashboard

- FocusSession logging: label picker, Pomodoro countdown, stopwatch.
- Community leaderboard query and display.
- Personal analytics: Recharts bar/line chart from FocusSession records.

Phase 6 — Study Circles (Most Complex)

- Socket.io server: room lifecycle, join/leave, cleanup on server restart.
- WebRTC signalling relay: offer/answer/ICE via Socket.io events.
- Voice + video in UI: RTCPeerConnection per peer pair, mute/camera controls.
- Screen sharing: getDisplayMedia, one active share per room.
- Text chat: Socket.io broadcast, ephemeral (memory only).
- Excalidraw whiteboard: embed component, emit onChange deltas, apply remote deltas.
- Snapshot: exportToBlob() → R2 upload → VaultItem creation.
- Open Relay TURN: configure ICE servers in RTCPeerConnection.

Phase 7 — Polish & Deploy

- Admin dashboard: global view, force-delete.
- Mobile responsiveness audit across all pages.
- Empty states, error states, loading skeletons on all key views.
- Environment config: .env validation, CORS hardening, production build.
- Deploy: Next.js to Vercel, Socket.io server to Railway/Render, Supabase production project, R2 production bucket.

11. Environment Variables & Deployment

Variable	Purpose
NEXT_PUBLIC_SUPABASE_URL	Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Supabase anon key (safe to expose client-side)
SUPABASE_SERVICE_ROLE_KEY	Supabase service role key (server-side only, admin operations)
GOOGLE_CLIENT_ID	Google OAuth client ID (also set in Supabase dashboard)
GOOGLE_CLIENT_SECRET	Google OAuth client secret (also set in Supabase dashboard)
R2_ACCOUNT_ID	Cloudflare account ID
R2_ACCESS_KEY_ID	R2 access key
R2_SECRET_ACCESS_KEY	R2 secret key
R2_BUCKET_NAME	R2 bucket name
R2_PUBLIC_ENDPOINT	R2 S3-compatible endpoint URL
NEXT_PUBLIC_SOCKET_URL	Socket.io server URL (used by React client to connect)
CLIENT_URL	Next.js app URL (used by Socket.io server for CORS whitelist)

Service	Recommended Host	Notes
Next.js App	Vercel	Free tier. Automatic deployments from Git.
Socket.io Server	Railway or Render	Free tier. ~50-line Node.js server.js file.
Database	Supabase (built-in)	Managed PostgreSQL. Backups included.
File Storage	Cloudflare R2	10 GB free. Zero egress fees.
TURN Server	Open Relay by Metered	Free tier. No credit card.